# Programowanie obliczeń komputerowych

# Dodatek: Integracja oprogramowania języka C/C++ w środowisku skryptowym Python

# Interfejs programowania CPython API

- Umożliwia programistom tworzenie modułów dla Python z kodem binarnym z języka C/C++ (lub innym np. Java z użyciem PyJ API)
- **<pyroot>\include**: podkatalog z plikami nagłówkowymi *.h zawierającymi deklarację interfejsu API dla języka C/C++

Uwaga: Ścieżkę tą należy dodać do środowiska IDE (np. vstudio)

- **<pyroot>\libs**: podkatalog z plikami binarnymi *.lib zawierającymi implementację lub import interfejsu API dla języka C/C++

Uwaga: Pliki *.h zawierają dyrektywę pragmatyczną nazw z p/w ścieżki

- **<pyroot>\DLLs**: podkatalog z modułami *.pyd oraz *.dll – może też zawierać główną implementację interfejsu API dla języka C/C++

Uwaga: Można opcjonalnie dodać tą ścieżkę do zmiennej PATH

- **<pyroot>\Doc**: podkatalog z dokumentacją *.chm interfejsu API
- Dodatkowe narzędzia dla programistów mogą się znajdować w podkatalogach: **<pyroot>\Tools** oraz **<pyroot>\Tools\Scripts**

# Moduły oparte o CPython2 API

- W Visual Studio (lub innym IDE) moduły traktowane jako aplikacje dla środowiska Python muszą być tworzone w projekcie typu "C++ - Win32 - DLL Library" z podzespołem *.pyd jako celem, przy czym
- należy włączyć do projektu ścieżkę dostępu do nagłówkowych plików API, np.: "c:\python27\include", takich jak "python.h"
- oraz do plików bibl., np.: "c:\python27\libs", jak "python27.lib"

Uwaga: Nazw plików nie trzeba jawnie podawać (są w pliku python.h)

- należy w projekcie zdefiniować makro "PY_MODULE_ =XYZ" zawierające główną nazwę ‘XYZ’ celu projektu, np.: "ptntest1"
- należy wstawić do projektu plik *.def eksportujący inicjalizacyjną funkcję ‘initXYZ’ do zarejestrowania publicznych metod modułu

Uwaga: Alternatywnie można też użyć specyfikator **`__declspec(dllexport)`**

- W projekcie wymagany jest typ kodowania znaków MBCS
- Kody źródłowe można pisać w plikach *.c|cpp dla języków C, C++
- W starszych wersjach Python moduł mógł mieć nazwę *.dll

# Użycie własnych modułów Python

- Po utworzeniu docelowego modułu DLL *.pyd dla Python można przejść w konsoli do powłoki z interpreterem i użyć ten moduł, np.:

```
> set PYTHONPATH=ptntest1\Release
> python
>>> import ptntest1
>>> ptntest1.Main()
```

**zmienna z listą (;) ścieżek względem katalogu bieżącego w systemie – może też zawierać kropkę lub wprost ścieżkę bezwzględną**

- Zmienna p/w dołączona jest do ścieżek poszukiwań `sys.path` przed ścieżkami systemowymi środowiska, takimi jak **<pyroot>\Lib**
- Import modułu woła funkcję init, co daje punkt wejścia do jego kodu
- Dzięki temu można wprost wywoływać metody z rejestru modułu
- W rezultacie powstaje aplikacja hybrydowa zawierająca skrypt źródłowy języka Python oraz kod binarny języka C/C++
- Przetwarzanie tej aplikacji zachodzi w środowisku Python, co daje dowolne rozszerzanie jego funkcjonalności (ang. *python-extending*)

# Przykład implementacji modułu Python

```
// pliki naglowkowe standardu jezyka C/C++
// oraz interfejsu API srodowiska Python
// dla jezyka C/C++:
#include <cstdlib>
#include <cstdio>
#include "Python.h"
using namespace std;
```

**uwaga: przykłady dotyczą jedynie API dla Python2 – język Python3 ma nieco zmienione API**

# Przykład implementacji modułu Python

```
// (glowna) funkcja publiczna modulu:
//
// jest to zwykla funkcja statyczna modulu a jej nazwa
// moze byc dowolna, funkcja ta moze byc wywolana ze skryptu
// pythona z p/n parametrami przekazanymi w punkcie
// wywolania ( self - obiekt tej funkcji, args - obiekt listy jej argumentów )
//
static PyObject *_main_(PyObject *self, PyObject *args)
{
 printf("Witaj swiecie\n"); // mozna tu korzystac ze biblioteki std

 Py_INCREF(Py_None); // powrot ze funkcji modulu do skryptu
 return Py_None;
}
```

**funkcja zwraca obiekt braku rezultatu o wartości None z inkrementacją licznika dowiązań**

# Przykład implementacji modułu Python

```
// zarejestrowanie listy metod modulu:
//
// jest to rejestr wszystkich p/w publicznych funkcji
// nalezacych do modulu i dostepnych w srodowisku
// skryptowym pythona
//
static PyMethodDef _rejestr_[] = {
    {"Main", _main_, 1, "Main() doc string"},
    {NULL, NULL}
};
```

zamiast wartości 1 może tu być makrosymbol METH_VARARGS który oznacza dowolną listę argumentów we Main(…)

uwaga: w przypadku braku listy args stosujemy makrosymbol METH_NOARGS ale wtedy wymagana jest konwersja _main_ do typu PyCFunction

# Przykład implementacji modułu Python

```
// funkcja inicjujaca modul z punktem wejscia do niego:
//
// jest to specjalna funkcja eksportowana ze modulu na zewnatrz
// jako funkcja jezyka C, eksportowanie moze odbywac sie poprzez
// opcje linkera /export lub specyfikator __declspec(dllexport)
// lub poprzez plik *.def w sekcji EXPORTS, zgodnie ze wymaganiami
// srodowiska python nazwa zewnetrzna p/n funkcji bedacej punktem
// wejscia srodowiska do modulu musi miec postac: initMODULE,
// gdzie 'MODULE' to nazwa utworzonego w projekcie pliku binarnego
// z modulem DLL (np. MODULE.pyd), nazwa ta jest przypisana do p/n
// dowolnej nazwy w pliku *.def dolaczonym do projektu
//
extern "C" void _init_()
{
 Py_InitModule(PY_MODULE_, _rejestr_); // PY_MODULE_ to makro zawierajace nazwe modulu DLL
}
```

```
EXPORTS
    initptntest1=_init_
```

# Programy oparte o CPython2 API

- Interfejs API środowiska Python pozwala również na wbudowanie interpretera języka Python i interpretację skryptów bezpośrednio w programie, a podanych jako literalny tekst lub odczytanych z pliku
- Program może ponadto pełnić rolę głównego modułu i zawierać definicje funkcji dołączonych do środowiska Python, co może być używane w skrypcie podobnie jak dla innych modułów
- W Visual Studio programy *.exe ze wbudowanym interpreterem (ang. *python-embedding*), tj. aplikacje skryptowe, są tworzone w projekcie typu "C++ - Win32 – Console Application", przy czym
- należy włączyć do projektu ścieżkę dostępu do nagłówkowych plików API, np.: "c:\python27\include", takich jak "python.h"
- oraz do plików bibl., np.: "c:\python27\libs", jak "python27.lib"

Uwaga: Nazw plików nie trzeba jawnie podawać (są w pliku python.h)

- W projekcie wymagany jest typ kodowania znaków MBCS
- Kody źródłowe można pisać w plikach *.c|cpp dla języka C/C++

# Przykład implementacji programu Python

```
// pliki naglowkowe standardu C/C++ oraz interfejsu CPython API:
#include <cstdlib>
#include <cstdio>
#include "Python.h"
using namespace std;

// glowna funkcja skryptu Python:

static PyObject *_main_(PyObject *self, PyObject *args)
{
 printf("witaj swiecie\n");
 Py_INCREF(Py_None);
 return Py_None;
}

// rejestracja i inicjacja funkcji Main:

static PyMethodDef _rejestr_[] = {
    {"Main", _main_, 1, "Main() doc string"},
    {NULL, NULL}
};
```

# Przykład implementacji programu Python

```
void _init_()
{
 Py_InitModule("Program", _rejestr_);
}

// glowna funkcja programu:

int main()
{
 Py_Initialize(); // zainicjowanie interpretera Python
 _init_(); // jawne wywolanie funkcji dolaczajacej funkcje Main do srodowiska
 PyRun_SimpleString("from Program import Main\n"
                    "Main()"); // interpretacja skryptu literalnego
 Py_Finalize(); // zamkniecie interpretera Python

 return 0;
}
```

- Po utworzeniu docelowego programu EXE ze Python można przejść w konsoli do katalogu Release i uruchomić plik, np.: ptntest2.exe

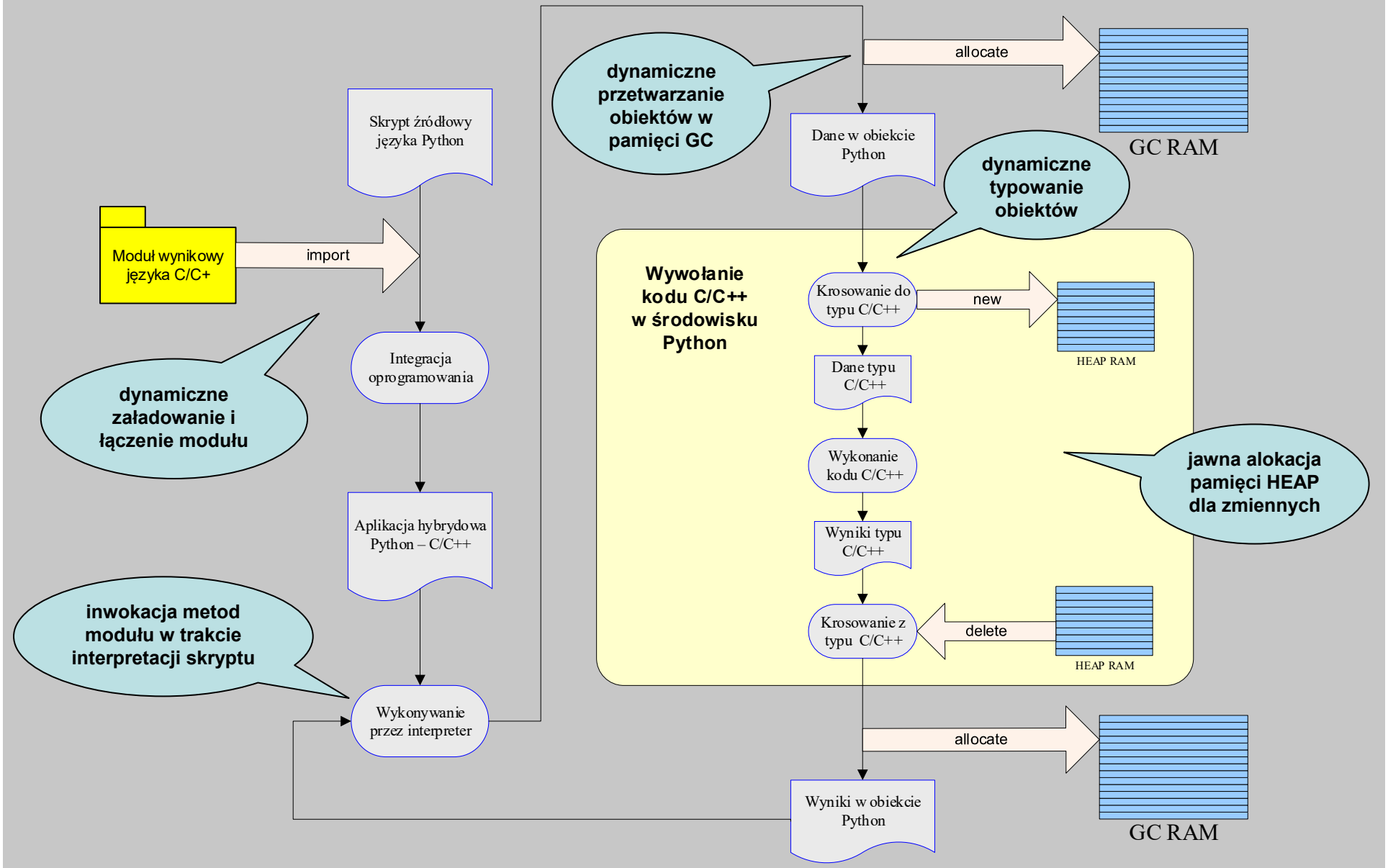
# Tworzenie obwiązań Python dla kodu C/C++

- Obiekty danych jako argumenty wywołania metod modułu DLL posiadają pamięć (GC) i strukturę charakterystyczną dla środowiska Python – mogą to być obiekty krotki, listy, a nawet prostych typów
- Na przykład utworzony docelowy moduł *.pyd można w powłoce z interpreterem następująco użyć:

```
>>> from ptntest1 import *
>>> SortArray(RandArray(10,987654321),True)
```

- W aplikacji hybrydowej zawierającej obwiązania (ang. *bindings*) Python dla kodu C/C++ przesyłanie danych z skryptu Python do kodu C/C++ i odwrotnie wymaga krosowania (ang. *marshalling*) danych przez barierę międzyjęzykową
- W takiej transformacji dane są migrowane pomiędzy odmiennymi strukturami obydwu języków z uwzględnieniem specyficznego dla języków zarządzania pamięcią, typizacji danych i ich mutacyjności

# Krosowanie danych w Python bindings



# Przykład implementacji obwiązań Python

```
// pliki naglowkowe standardu jezyka C/C++ oraz CPython API:
#include <cstdlib>
#include <ctime>
#include <cmath>
#include <iostream>
#include "Python.h"
using namespace std;

// pomocnicza funkcja do porownywania danych:
static int dblcmp(const void *arg1, const void *arg2)
{
 double a = *reinterpret_cast<const double*>(arg1);
 double b = *reinterpret_cast<const double*>(arg2);
 return a<b ? -1 : a>b ? +1 : 0;
}
```

# Przykład implementacji obwiązań Python

```
// podstawowa metoda modulu do sortowania danych:

static PyObject *SortArray(PyObject *self, PyObject *args)
{
 PyObject *list; // krosowanie danych z obiektu krotki Python (argumenty funkcji) do zmiennych typu C/C++
 bool verbose = false;
 if (!PyArg_ParseTuple(args, "O|b", &list, &verbose)) return NULL;

 int n = PyList_Size(list);
 if (verbose)
 cout << "n = " << n << endl;

 double *A = new double[n]; // alokacja pamieci C/C++ (HEAP) dla pomocniczych struktur danych

 for (int i=0; i<n; i++) { // dalsze krosowanie danych z obiektu listy Python do struktur typu C/C++
  double x = 0.0;
  PyObject *item = PyList_GetItem(list, i);
  if (PyFloat_Check(item) || PyInt_Check(item)) {
   x = PyFloat_AsDouble(item);
   if (verbose)
   cout << i << ") x = " << x << endl;
  }
  A[i] = x;
 }

 qsort(A,n,sizeof(double),dblcmp); // wykonanie kodu C/C++ dla algorytmu "Quick Sort"

 list = PyList_New(n); // alokacja pamieci Python (GC) dla wynikow obliczen

 for (int i=0; i<n; i++) { // krosowanie wynikow z struktur typu C/C++ do obiektu listy Python
  double y = A[i];
  if (verbose)
  cout << i << ") y = " << y << endl;
  PyObject *item = PyFloat_FromDouble(y);
  PyList_SetItem(list, i, item);
 }

 delete[] A; // dealokacja pamieci C/C++

 return Py_BuildValue("O", list); // utworzenie obiektu wartosci Python dla rezultatu funkcji
}
```

# Przykład implementacji obwiązań Python

```
// dodatkowa metoda modulu do generowania danych:

static PyObject *RandArray(PyObject *self, PyObject *args)
{
 int n;
 unsigned seed = (int)time(NULL);

 if (!PyArg_ParseTuple(args, "i|I", &n, &seed)) return NULL;

 srand(seed); // wykonanie kodu C/C++ do pseudolosowania danych
 PyObject *list = PyList_New(n);
 for (int i=0; i<n; i++) {
  double y = rand();
  PyObject *item = PyFloat_FromDouble((double)y);
  PyList_SetItem(list, i, item);
 }

 return Py_BuildValue("O", list);
}
```

# Przykład implementacji obwiązań Python

```
// rejestrowanie metod modulu i jego zainicjowanie:

static PyMethodDef _rejestr_[] = {
    {"SortArray", (PyCFunction)SortArray, METH_VARARGS, "Call the 'qsort' C/C++ routine"},
    {"RandArray", (PyCFunction)RandArray, METH_VARARGS, "Call the 'rand' C/C++ routine"},
    {NULL, NULL}
};

extern "C" void _init_()
{
    Py_InitModule(PY_MODULE_, _rejestr_);
}
```